DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
23CSR306 JAVA PROGRAMMING — ASSIGNMENT II: OOP MINI PROJECT

2. ASSIGNMENT QUESTION (EXACT SPECIFICATION)

Specification 1: Inventory and Billing System

Problem Statement:

Design and implement a Java console application for an Inventory and Billing System that handles product purchases, sales, customer types, and a reward points system for premium customers. The application should demonstrate object-oriented principles (encapsulation, inheritance, aggregation, polymorphism) and maintain accurate stock quantities.

Class Requirements:

Product, ProductTaxCategory, Customer, PremiumCustomer, SaleDetail, Sale, PurchaseDetail, Purchase

Business Rules:

1. Stock must be verified before confirming a sale. If insufficient stock is available, the sale should not proceed.
2. Stock quantity must be updated immediately after a sale or purchase.
3. Premium customers earn 1 point for every Rs.100 spent (after discounts, before tax). Once points reach 200 or more, a discount of Rs.10 per 100 points is applied, and redeemed points are deducted.
4. Regular customers do not receive points or discounts.
5. Sale and purchase details should link directly to the associated products.

Menu Options:

Add Product / Add Customer / Make a Purchase / Make a Sale / Display Products / Exit

Expected Output:

- Sale bill with product list, quantities, prices, taxes, discounts (if applicable), total amount and updated reward points (for premium customers).
- Updated stock after every sale or purchase.

OOP ARCHITECTURE & DESIGN MAPPING

OOP Principle	Implementation in Solution Architecture
Encapsulation	All entity classes (Product, Customer, Sale, Purchase, etc.) declare private fields with strictly validated getters/setters. Domain invariants (non-negative stock, non-zero pricing) are enforced internally.
Inheritance	PremiumCustomer extends Customer. It inherits baseline customer identity (customerId, name, contact) and extends functionality with rewardPoints and membershipTier.
Polymorphism	Customer declares polymorphic methods calculateDiscount(), calculatePointsEarned(), and redeemPointsForDiscount(). PremiumCustomer overrides them to apply business logic (Rs.10/100 pts and 1 pt/Rs.100). Dynamic dispatch handles billing uniformly.
Aggregation	SaleDetail and PurchaseDetail maintain direct references to independent Product objects (Business Rule 5). Sale maintains an aggregated reference to Customer.
Composition	Sale contains a List<SaleDetail> whose lifecycle is governed by the sale transaction. Purchase contains a List<PurchaseDetail>.
Exception Handling	Custom checked exceptions (InsufficientStockException, ProductNotFoundException, etc.) enforce business rules and prevent system crashes.

3. JAVA SOURCE CODE (PRINTED FROM ECLIPSE IDE)

Package hierarchy and complete class implementations. All classes include private data members, encapsulation, inheritance, aggregation, polymorphism, and custom exception handling.

[Eclipse IDE Java Editor] Package: com.inventory.model | File: ProductTaxCategory.java

```java
package com.inventory.model;

import java.io.Serializable;

/**
 * Represents the tax category applicable to products in the inventory system.
 * Encapsulates GST/tax rates and category metadata.
 */
public class ProductTaxCategory implements Serializable {
    private static final long serialVersionUID = 1L;

    private String categoryId;
    private String categoryName;
    private double gstRate; // in percentage, e.g., 5.0, 12.0, 18.0

    // Standard pre-defined tax categories
    public static final ProductTaxCategory EXEMPT = new ProductTaxCategory("TC01", "Exempt (0%)", 0.0);
    public static final ProductTaxCategory ESSENTIAL = new ProductTaxCategory("TC02", "Essential (5%)", 5.0);
    public static final ProductTaxCategory STANDARD = new ProductTaxCategory("TC03", "Standard (12%)", 12.0);
    public static final ProductTaxCategory ELECTRONICS = new ProductTaxCategory("TC04", "Electronics (18%)", 18.0);
    public static final ProductTaxCategory LUXURY = new ProductTaxCategory("TC05", "Luxury (28%)", 28.0);

    public ProductTaxCategory() {
    }

    public ProductTaxCategory(String categoryId, String categoryName, double gstRate) {
        this.categoryId = categoryId;
        this.categoryName = categoryName;
        this.gstRate = Math.max(0.0, gstRate);
    }

    public String getCategoryId() {
        return categoryId;
    }

    public void setCategoryId(String categoryId) {
        this.categoryId = categoryId;
    }

    public String getCategoryName() {
        return categoryName;
    }

    public void setCategoryName(String categoryName) {
        this.categoryName = categoryName;
    }

    public double getGstRate() {
        return gstRate;
    }

    public void setGstRate(double gstRate) {
        this.gstRate = Math.max(0.0, gstRate);
    }

    @Override
    public String toString() {
        return categoryName + " [" + gstRate + "%]";
    }
}

```

[Eclipse IDE Java Editor] Package: com.inventory.model | File: Product.java

```java
package com.inventory.model;

import com.inventory.exception.InsufficientStockException;
import com.inventory.exception.InvalidInputException;
import java.io.Serializable;

/**
 * Represents an inventory product.
 * Implements encapsulation and links directly to a ProductTaxCategory.
 */
public class Product implements Serializable {
    private static final long serialVersionUID = 1L;

```

(continued) com.inventory.model.Product.java

```
14      private String productId;
15      private String name;
16      private double price;
17      private int stockQuantity;
18      private ProductTaxCategory taxCategory;
19
20      public Product() {
21      }
22
23      public Product(String productId, String name, double price, int stockQuantity, ProductTaxCategory taxCategory)
24              throws InvalidInputException {
25          if (productId == null || productId.trim().isEmpty()) {
26              throw new InvalidInputException("Product ID cannot be null or empty.");
27          }
28          if (name == null || name.trim().isEmpty()) {
29              throw new InvalidInputException("Product Name cannot be null or empty.");
30          }
31          if (price < 0) {
32              throw new InvalidInputException("Product price cannot be negative.");
33          }
34          if (stockQuantity < 0) {
35              throw new InvalidInputException("Stock quantity cannot be negative.");
36          }
37          this.productId = productId.trim().toUpperCase();
38          this.name = name.trim();
39          this.price = price;
40          this.stockQuantity = stockQuantity;
41          this.taxCategory = (taxCategory != null) ? taxCategory : ProductTaxCategory.STANDARD;
42      }
43
44      public String getProductId() {
45          return productId;
46      }
47
48      public void setProductId(String productId) throws InvalidInputException {
49          if (productId == null || productId.trim().isEmpty()) {
50              throw new InvalidInputException("Product ID cannot be empty.");
51          }
52          this.productId = productId.trim().toUpperCase();
53      }
54
55      public String getName() {
56          return name;
57      }
58
59      public void setName(String name) throws InvalidInputException {
60          if (name == null || name.trim().isEmpty()) {
61              throw new InvalidInputException("Product name cannot be empty.");
62          }
63          this.name = name.trim();
64      }
65
66      public double getPrice() {
67          return price;
68      }
69
70      public void setPrice(double price) throws InvalidInputException {
71          if (price < 0) {
72              throw new InvalidInputException("Price cannot be negative.");
73          }
74          this.price = price;
75      }
76
77      public int getStockQuantity() {
78          return stockQuantity;
79      }
80
81      public void setStockQuantity(int stockQuantity) throws InvalidInputException {
82          if (stockQuantity < 0) {
83              throw new InvalidInputException("Stock quantity cannot be negative.");
84          }
85          this.stockQuantity = stockQuantity;
86      }
87
88      public ProductTaxCategory getTaxCategory() {
89          return taxCategory;
90      }
91
92      public void setTaxCategory(ProductTaxCategory taxCategory) {
93          this.taxCategory = (taxCategory != null) ? taxCategory : ProductTaxCategory.STANDARD;
94      }
95
```

(continued) com.inventory.model.Product.java

```
96      /**
97       * Increases stock quantity immediately after a purchase.
98       * Enforces Business Rule 2.
99       */
100     public synchronized void addStock(int quantity) throws InvalidInputException {
101         if (quantity <= 0) {
102             throw new InvalidInputException("Quantity to add must be positive.");
103         }
104         this.stockQuantity += quantity;
105     }
106
107     /**
108      * Verifies and decreases stock quantity immediately after a sale.
109      * Enforces Business Rules 1 & 2.
110      */
111     public synchronized void deductStock(int quantity) throws InsufficientStockException, InvalidInputException {
112         if (quantity <= 0) {
113             throw new InvalidInputException("Quantity to deduct must be positive.");
114         }
115         if (this.stockQuantity < quantity) {
116             throw new InsufficientStockException(this.productId, quantity, this.stockQuantity);
117         }
118         this.stockQuantity -= quantity;
119     }
120
121     /**
122      * Calculates tax amount per unit of this product based on its tax category.
123      */
124     public double getTaxAmountPerUnit() {
125         if (taxCategory == null) return 0.0;
126         return (price * taxCategory.getGstRate()) / 100.0;
127     }
128
129     @Override
130     public String toString() {
131         return String.format("%-8s | %-22s | Rs. %-9.2f | Stock: %-5d | Tax: %s",
132                 productId, name, price, stockQuantity, (taxCategory != null ? taxCategory.toString() : "None"));
133     }
134 }
135
```

[Eclipse IDE Java Editor] Package: com.inventory.model | File: Customer.java

```
1   package com.inventory.model;
2
3   import com.inventory.exception.InvalidInputException;
4   import java.io.Serializable;
5
6   /**
7    * Base class representing a customer in the inventory and billing system.
8    * Demonstrates Encapsulation and acts as the parent for polymorphic customer types.
9    */
10  public class Customer implements Serializable {
11      private static final long serialVersionUID = 1L;
12
13      private String customerId;
14      private String name;
15      private String contactNumber;
16      private String email;
17
18      public Customer() {
19      }
20
21      public Customer(String customerId, String name, String contactNumber, String email)
22              throws InvalidInputException {
23          if (customerId == null || customerId.trim().isEmpty()) {
24              throw new InvalidInputException("Customer ID cannot be empty.");
25          }
26          if (name == null || name.trim().isEmpty()) {
27              throw new InvalidInputException("Customer Name cannot be empty.");
28          }
29          this.customerId = customerId.trim().toUpperCase();
30          this.name = name.trim();
31          this.contactNumber = (contactNumber != null) ? contactNumber.trim() : "";
32          this.email = (email != null) ? email.trim() : "";
33      }
34
35      public String getCustomerId() {
36          return customerId;
37      }
38
39      public void setCustomerId(String customerId) throws InvalidInputException {
```

(continued) com.inventory.model.Customer.java

```
        if (customerId == null || customerId.trim().isEmpty()) {
            throw new InvalidInputException("Customer ID cannot be empty.");
        }
        this.customerId = customerId.trim().toUpperCase();
    }

    public String getName() {
        return name;
    }

    public void setName(String name) throws InvalidInputException {
        if (name == null || name.trim().isEmpty()) {
            throw new InvalidInputException("Customer Name cannot be empty.");
        }
        this.name = name.trim();
    }

    public String getContactNumber() {
        return contactNumber;
    }

    public void setContactNumber(String contactNumber) {
        this.contactNumber = (contactNumber != null) ? contactNumber.trim() : "";
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = (email != null) ? email.trim() : "";
    }

    /**
     * Polymorphic method returning the customer category.
     */
    public String getCustomerType() {
        return "Regular";
    }

    /**
     * Polymorphic method calculating eligible discount before tax.
     * Business Rule 4: Regular customers do not receive points or discounts.
     */
    public double calculateDiscount(double spentBeforeTax) {
        return 0.0;
    }

    /**
     * Polymorphic method calculating earned reward points.
     * Business Rule 4: Regular customers do not receive points or discounts.
     */
    public int calculatePointsEarned(double amountAfterDiscountBeforeTax) {
        return 0;
    }

    /**
     * Deducts redeemed reward points from the customer balance.
     * No-op for regular customers.
     */
    public int redeemPointsForDiscount(double discountApplied) {
        return 0;
    }

    /**
     * Returns current reward points. Regular customers have 0.
     */
    public int getRewardPoints() {
        return 0;
    }

    /**
     * Adds reward points to customer account. No-op for regular customers.
     */
    public void addRewardPoints(int points) {
        // Regular customers do not collect points
    }

    @Override
    public String toString() {
        return String.format("%-8s | %-20s | %-12s | %-10s | %s",
                customerId, name, contactNumber, getCustomerType(), email);
```

(continued) com.inventory.model.Customer.java

```
    }
}

```

[Eclipse IDE Java Editor] Package: com.inventory.model | File: PremiumCustomer.java

```
package com.inventory.model;

import com.inventory.exception.InvalidInputException;

/**
 * Represents a premium customer in the inventory system.
 * Extends Customer to demonstrate Inheritance and Polymorphism.
 * Enforces Business Rule 3:
 * - Earns 1 point for every Rs. 100 spent (after discounts, before tax).
 * - When points reach 200 or more, Rs. 10 discount per 100 points is applied, and points are deducted.
 */
public class PremiumCustomer extends Customer {
    private static final long serialVersionUID = 1L;

    private int rewardPoints;
    private String membershipTier;

    public PremiumCustomer() {
        super();
        this.rewardPoints = 0;
        this.membershipTier = "Gold";
    }

    public PremiumCustomer(String customerId, String name, String contactNumber, String email, int initialPoints)
            throws InvalidInputException {
        super(customerId, name, contactNumber, email);
        if (initialPoints < 0) {
            throw new InvalidInputException("Initial reward points cannot be negative.");
        }
        this.rewardPoints = initialPoints;
        this.membershipTier = determineTier(initialPoints);
    }

    private String determineTier(int points) {
        if (points >= 1000) return "Platinum";
        if (points >= 500) return "Gold";
        return "Silver";
    }

    @Override
    public String getCustomerType() {
        return "Premium (" + membershipTier + ")";
    }

    @Override
    public int getRewardPoints() {
        return rewardPoints;
    }

    public void setRewardPoints(int rewardPoints) throws InvalidInputException {
        if (rewardPoints < 0) {
            throw new InvalidInputException("Reward points cannot be negative.");
        }
        this.rewardPoints = rewardPoints;
        this.membershipTier = determineTier(rewardPoints);
    }

    public String getMembershipTier() {
        return membershipTier;
    }

    public void setMembershipTier(String membershipTier) {
        this.membershipTier = membershipTier;
    }

    /**
     * Polymorphic implementation of discount calculation.
     * Rule 3: Once points reach 200 or more, a discount of Rs. 10 per 100 points is applied.
     */
    @Override
    public double calculateDiscount(double spentBeforeTax) {
        if (this.rewardPoints < 200 || spentBeforeTax <= 0) {
            return 0.0;
        }
        int eligibleBlocks = this.rewardPoints / 100;
        double potentialDiscount = eligibleBlocks * 10.0;
```

(continued) com.inventory.model.PremiumCustomer.java

```
77
78          // Discount cannot exceed the gross amount before tax
79          if (potentialDiscount > spentBeforeTax) {
80              int applicableBlocks = (int) (spentBeforeTax / 10.0);
81              return applicableBlocks * 10.0;
82          }
83          return potentialDiscount;
84      }
85
86      /**
87       * Polymorphic calculation of reward points earned.
88       * Rule 3: Earns 1 point for every Rs. 100 spent (after discounts, before tax).
89       */
90      @Override
91      public int calculatePointsEarned(double amountAfterDiscountBeforeTax) {
92          if (amountAfterDiscountBeforeTax < 100.0) {
93              return 0;
94          }
95          return (int) (amountAfterDiscountBeforeTax / 100.0);
96      }
97
98      /**
99       * Deducts points corresponding to the applied discount.
100      * Rule 3: Redeemed points are deducted (100 points per Rs. 10 discount).
101      */
102     @Override
103     public int redeemPointsForDiscount(double discountApplied) {
104         if (discountApplied <= 0) {
105             return 0;
106         }
107         int pointsToDeduct = ((int) Math.round(discountApplied / 10.0)) * 100;
108         if (pointsToDeduct > this.rewardPoints) {
109             pointsToDeduct = this.rewardPoints;
110         }
111         this.rewardPoints -= pointsToDeduct;
112         this.membershipTier = determineTier(this.rewardPoints);
113         return pointsToDeduct;
114     }
115
116     /**
117      * Updates customer reward points with newly earned points.
118      */
119     @Override
120     public void addRewardPoints(int points) {
121         if (points > 0) {
122             this.rewardPoints += points;
123             this.membershipTier = determineTier(this.rewardPoints);
124         }
125     }
126
127     @Override
128     public String toString() {
129         return String.format("%-8s | %-20s | %-12s | %-10s | %-18s | Points: %d",
130                 getCustomerId(), getName(), getContactNumber(), "Premium", getEmail(), rewardPoints);
131     }
132 }
133
```

[Eclipse IDE Java Editor] Package: com.inventory.model | File: PurchaseDetail.java

```
1  package com.inventory.model;
2
3  import com.inventory.exception.InvalidInputException;
4  import java.io.Serializable;
5
6  /**
7   * Represents an individual line item in a restock/purchase order.
8   * Demonstrates Aggregation by directly linking to an associated Product (Business Rule 5).
9   */
10 public class PurchaseDetail implements Serializable {
11     private static final long serialVersionUID = 1L;
12
13     private Product product;
14     private int quantity;
15     private double unitCostPrice;
16
17     public PurchaseDetail() {
18     }
19
20     public PurchaseDetail(Product product, int quantity, double unitCostPrice) throws InvalidInputException {
21         if (product == null) {
22             throw new InvalidInputException("Associated product cannot be null.");
```

(continued) com.inventory.model.PurchaseDetail.java

```
        }
        if (quantity <= 0) {
            throw new InvalidInputException("Purchase quantity must be greater than zero.");
        }
        if (unitCostPrice < 0) {
            throw new InvalidInputException("Unit cost price cannot be negative.");
        }
        this.product = product;
        this.quantity = quantity;
        this.unitCostPrice = unitCostPrice;
    }

    public Product getProduct() {
        return product;
    }

    public void setProduct(Product product) throws InvalidInputException {
        if (product == null) {
            throw new InvalidInputException("Associated product cannot be null.");
        }
        this.product = product;
    }

    public int getQuantity() {
        return quantity;
    }

    public void setQuantity(int quantity) throws InvalidInputException {
        if (quantity <= 0) {
            throw new InvalidInputException("Purchase quantity must be greater than zero.");
        }
        this.quantity = quantity;
    }

    public double getUnitCostPrice() {
        return unitCostPrice;
    }

    public void setUnitCostPrice(double unitCostPrice) throws InvalidInputException {
        if (unitCostPrice < 0) {
            throw new InvalidInputException("Unit cost price cannot be negative.");
        }
        this.unitCostPrice = unitCostPrice;
    }

    public double getSubtotal() {
        return quantity * unitCostPrice;
    }

    @Override
    public String toString() {
        return String.format("%-8s | %-20s | %5d units @ Rs. %-8.2f = Rs. %-10.2f",
                product.getProductId(), product.getName(), quantity, unitCostPrice, getSubtotal());
    }
}

```

[Eclipse IDE Java Editor] Package: com.inventory.model | File: Purchase.java

```
package com.inventory.model;

import com.inventory.exception.InvalidInputException;
import java.io.Serializable;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/**
 * Represents a purchase/restocking transaction from a supplier.
 * Demonstrates Composition by owning a list of PurchaseDetail items.
 * Enforces Business Rule 2: Stock quantity must be updated immediately after a purchase.
 */
public class Purchase implements Serializable {
    private static final long serialVersionUID = 1L;

    private String purchaseId;
    private LocalDateTime purchaseDate;
    private String supplierName;
    private List<PurchaseDetail> purchaseDetails;
    private double totalAmount;
```

(continued) com.inventory.model.Purchase.java

```
    private boolean processed;

    public Purchase() {
        this.purchaseDetails = new ArrayList<>();
        this.purchaseDate = LocalDateTime.now();
        this.processed = false;
    }

    public Purchase(String purchaseId, String supplierName) throws InvalidInputException {
        if (purchaseId == null || purchaseId.trim().isEmpty()) {
            throw new InvalidInputException("Purchase ID cannot be empty.");
        }
        if (supplierName == null || supplierName.trim().isEmpty()) {
            throw new InvalidInputException("Supplier name cannot be empty.");
        }
        this.purchaseId = purchaseId.trim().toUpperCase();
        this.supplierName = supplierName.trim();
        this.purchaseDate = LocalDateTime.now();
        this.purchaseDetails = new ArrayList<>();
        this.totalAmount = 0.0;
        this.processed = false;
    }

    public String getPurchaseId() {
        return purchaseId;
    }

    public void setPurchaseId(String purchaseId) throws InvalidInputException {
        if (purchaseId == null || purchaseId.trim().isEmpty()) {
            throw new InvalidInputException("Purchase ID cannot be empty.");
        }
        this.purchaseId = purchaseId.trim().toUpperCase();
    }

    public LocalDateTime getPurchaseDate() {
        return purchaseDate;
    }

    public void setPurchaseDate(LocalDateTime purchaseDate) {
        this.purchaseDate = purchaseDate;
    }

    public String getSupplierName() {
        return supplierName;
    }

    public void setSupplierName(String supplierName) throws InvalidInputException {
        if (supplierName == null || supplierName.trim().isEmpty()) {
            throw new InvalidInputException("Supplier name cannot be empty.");
        }
        this.supplierName = supplierName.trim();
    }

    public List<PurchaseDetail> getPurchaseDetails() {
        return Collections.unmodifiableList(purchaseDetails);
    }

    public double getTotalAmount() {
        return totalAmount;
    }

    public boolean isProcessed() {
        return processed;
    }

    /**
     * Adds an item detail to the purchase.
     */
    public void addPurchaseDetail(PurchaseDetail detail) throws InvalidInputException {
        if (detail == null) {
            throw new InvalidInputException("Purchase detail cannot be null.");
        }
        if (processed) {
            throw new InvalidInputException("Cannot add details to an already processed purchase.");
        }
        purchaseDetails.add(detail);
        totalAmount += detail.getSubtotal();
    }

    /**
     * Confirms the purchase and immediately updates product stock.
     * Enforces Business Rule 2.
```

(continued) com.inventory.model.Purchase.java

```
     */
    public synchronized void processPurchase() throws InvalidInputException {
        if (processed) {
            throw new InvalidInputException("Purchase has already been processed.");
        }
        if (purchaseDetails.isEmpty()) {
            throw new InvalidInputException("Cannot process an empty purchase order.");
        }
        for (PurchaseDetail detail : purchaseDetails) {
            detail.getProduct().addStock(detail.getQuantity());
        }
        this.processed = true;
    }

    /**
     * Generates a detailed receipt for the purchase order.
     */
    public String getPurchaseReceipt() {
        DateTimeFormatter dtf = DateTimeFormatter.ofPattern("dd-MMM-yyyy HH:mm:ss");
        StringBuilder sb = new StringBuilder();
        sb.append("================================================================================\n");
        sb.append("                      PURCHASE / RESTOCK RECEIPT                                \n");
        sb.append("================================================================================\n");
        sb.append(String.format("Purchase ID : %-25s Date: %s\n", purchaseId, purchaseDate.format(dtf)));
        sb.append(String.format("Supplier    : %s\n", supplierName));
        sb.append("--------------------------------------------------------------------------------\n");
        sb.append(String.format("%-10s %-24s %8s %12s %14s\n", "Prod ID", "Product Name", "Qty", "Cost (Rs. )",
"Subtotal (Rs. )"));
        sb.append("--------------------------------------------------------------------------------\n");
        for (PurchaseDetail d : purchaseDetails) {
            sb.append(String.format("%-10s %-24s %8d %12.2f %14.2f\n",
                    d.getProduct().getProductId(),
                    d.getProduct().getName(),
                    d.getQuantity(),
                    d.getUnitCostPrice(),
                    d.getSubtotal()));
        }
        sb.append("--------------------------------------------------------------------------------\n");
        sb.append(String.format("TOTAL PURCHASE AMOUNT:                                  Rs. %14.2f\n", totalAmount));
        sb.append("--------------------------------------------------------------------------------\n");
        sb.append("UPDATED PRODUCT STOCK LEVELS:\n");
        for (PurchaseDetail d : purchaseDetails) {
            sb.append(String.format("  • %s (%s): Current Available Stock = %d units\n",
                    d.getProduct().getName(),
                    d.getProduct().getProductId(),
                    d.getProduct().getStockQuantity()));
        }
        sb.append("================================================================================\n");
        return sb.toString();
    }
}

```

[Eclipse IDE Java Editor] Package: com.inventory.model | File: SaleDetail.java

```
package com.inventory.model;

import com.inventory.exception.InvalidInputException;
import java.io.Serializable;

/**
 * Represents an individual line item in a sales invoice.
 * Demonstrates Aggregation by directly linking to an associated Product (Business Rule 5).
 */
public class SaleDetail implements Serializable {
    private static final long serialVersionUID = 1L;

    private Product product;
    private int quantity;
    private double unitPrice;
    private double taxRate; // in %

    public SaleDetail() {
    }

    public SaleDetail(Product product, int quantity) throws InvalidInputException {
        if (product == null) {
            throw new InvalidInputException("Associated product cannot be null.");
        }
        if (quantity <= 0) {
            throw new InvalidInputException("Sale quantity must be greater than zero.");
        }
        this.product = product;
```

(continued) com.inventory.model.SaleDetail.java

```
        this.quantity = quantity;
        this.unitPrice = product.getPrice();
        this.taxRate = (product.getTaxCategory() != null) ? product.getTaxCategory().getGstRate() : 0.0;
    }

    public SaleDetail(Product product, int quantity, double unitPrice) throws InvalidInputException {
        if (product == null) {
            throw new InvalidInputException("Associated product cannot be null.");
        }
        if (quantity <= 0) {
            throw new InvalidInputException("Sale quantity must be greater than zero.");
        }
        if (unitPrice < 0) {
            throw new InvalidInputException("Unit price cannot be negative.");
        }
        this.product = product;
        this.quantity = quantity;
        this.unitPrice = unitPrice;
        this.taxRate = (product.getTaxCategory() != null) ? product.getTaxCategory().getGstRate() : 0.0;
    }

    public Product getProduct() {
        return product;
    }

    public void setProduct(Product product) throws InvalidInputException {
        if (product == null) {
            throw new InvalidInputException("Associated product cannot be null.");
        }
        this.product = product;
        this.taxRate = (product.getTaxCategory() != null) ? product.getTaxCategory().getGstRate() : 0.0;
    }

    public int getQuantity() {
        return quantity;
    }

    public void setQuantity(int quantity) throws InvalidInputException {
        if (quantity <= 0) {
            throw new InvalidInputException("Quantity must be greater than zero.");
        }
        this.quantity = quantity;
    }

    public double getUnitPrice() {
        return unitPrice;
    }

    public void setUnitPrice(double unitPrice) throws InvalidInputException {
        if (unitPrice < 0) {
            throw new InvalidInputException("Unit price cannot be negative.");
        }
        this.unitPrice = unitPrice;
    }

    public double getTaxRate() {
        return taxRate;
    }

    /**
     * Calculates gross amount before tax and discount for this line item.
     */
    public double getGrossAmount() {
        return quantity * unitPrice;
    }

    /**
     * Calculates tax amount for this line item based on its tax rate.
     */
    public double getTaxAmount() {
        return (getGrossAmount() * taxRate) / 100.0;
    }

    /**
     * Returns gross amount plus tax for this line item.
     */
    public double getLineTotal() {
        return getGrossAmount() + getTaxAmount();
    }

    @Override
    public String toString() {
```

(continued) com.inventory.model.SaleDetail.java

```
        return String.format("%-8s | %-20s | %5d | Rs. %-8.2f | %4.1f%% | Rs. %-8.2f | Rs. %-9.2f",
                product.getProductId(), product.getName(), quantity, unitPrice, taxRate, getTaxAmount(), getLineTotal());
    }
}

```

[Eclipse IDE Java Editor] Package: com.inventory.model | File: Sale.java

```
package com.inventory.model;

import com.inventory.exception.InsufficientStockException;
import com.inventory.exception.InvalidInputException;
import java.io.Serializable;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/**
 * Represents a completed sales transaction and tax invoice.
 * Demonstrates:
 * - Aggregation: Customer reference (polymorphic dispatch for discounts and reward points)
 * - Composition: List of SaleDetail objects
 * - Business Rules:
 *   1. Atomic stock verification prior to confirmation.
 *   2. Immediate stock decrement upon confirmation.
 *   3. Reward points redemption and accrual for PremiumCustomer.
 *   4. Zero points and zero discount for regular Customer.
 *   5. Sale bill formatting displaying all required breakdown details.
 */
public class Sale implements Serializable {
    private static final long serialVersionUID = 1L;

    private String saleId;
    private LocalDateTime saleDate;
    private Customer customer;
    private List<SaleDetail> saleDetails;

    // Financial totals
    private double grossTotal;
    private double discountAmount;
    private int pointsRedeemed;
    private double subtotalAfterDiscount;
    private double totalTaxAmount;
    private double netTotal;

    // Loyalty points tracking
    private int pointsBeforeSale;
    private int pointsEarned;
    private int pointsAfterSale;

    private boolean processed;

    public Sale() {
        this.saleDetails = new ArrayList<>();
        this.saleDate = LocalDateTime.now();
        this.processed = false;
    }

    public Sale(String saleId, Customer customer) throws InvalidInputException {
        if (saleId == null || saleId.trim().isEmpty()) {
            throw new InvalidInputException("Sale ID cannot be empty.");
        }
        if (customer == null) {
            throw new InvalidInputException("Customer cannot be null for a sale.");
        }
        this.saleId = saleId.trim().toUpperCase();
        this.customer = customer;
        this.saleDate = LocalDateTime.now();
        this.saleDetails = new ArrayList<>();
        this.processed = false;
    }

    public String getSaleId() {
        return saleId;
    }

    public void setSaleId(String saleId) throws InvalidInputException {
        if (saleId == null || saleId.trim().isEmpty()) {
            throw new InvalidInputException("Sale ID cannot be empty.");
        }
```

(continued) com.inventory.model.Sale.java

```
75          this.saleId = saleId.trim().toUpperCase();
76      }
77
78      public LocalDateTime getSaleDate() {
79          return saleDate;
80      }
81
82      public void setSaleDate(LocalDateTime saleDate) {
83          this.saleDate = saleDate;
84      }
85
86      public Customer getCustomer() {
87          return customer;
88      }
89
90      public void setCustomer(Customer customer) throws InvalidInputException {
91          if (customer == null) {
92              throw new InvalidInputException("Customer cannot be null.");
93          }
94          this.customer = customer;
95      }
96
97      public List<SaleDetail> getSaleDetails() {
98          return Collections.unmodifiableList(saleDetails);
99      }
100
101     public double getGrossTotal() {
102         return grossTotal;
103     }
104
105     public double getDiscountAmount() {
106         return discountAmount;
107     }
108
109     public int getPointsRedeemed() {
110         return pointsRedeemed;
111     }
112
113     public double getSubtotalAfterDiscount() {
114         return subtotalAfterDiscount;
115     }
116
117     public double getTotalTaxAmount() {
118         return totalTaxAmount;
119     }
120
121     public double getNetTotal() {
122         return netTotal;
123     }
124
125     public int getPointsBeforeSale() {
126         return pointsBeforeSale;
127     }
128
129     public int getPointsEarned() {
130         return pointsEarned;
131     }
132
133     public int getPointsAfterSale() {
134         return pointsAfterSale;
135     }
136
137     public boolean isProcessed() {
138         return processed;
139     }
140
141     /**
142      * Adds an item to the sale before processing.
143      */
144     public void addSaleDetail(SaleDetail detail) throws InvalidInputException {
145         if (detail == null) {
146             throw new InvalidInputException("Sale detail cannot be null.");
147         }
148         if (processed) {
149             throw new InvalidInputException("Cannot add items to a finalized sale.");
150         }
151         saleDetails.add(detail);
152     }
153
154     /**
155      * Processes and finalizes the sale adhering to all business rules:
156      * 1. Verifies stock availability for ALL items before deducting any stock (Rule 1).
```

(continued) com.inventory.model.Sale.java

```
157         * 2. Calculates gross total across all sale details.
158         * 3. Calculates customer discount and points redemption polymorphically (Rules 3 & 4).
159         * 4. Deducts stock immediately upon confirmation (Rule 2).
160         * 5. Accrues new reward points for premium customer (Rule 3).
161         */
162        public synchronized void processSale() throws InsufficientStockException, InvalidInputException {
163            if (processed) {
164                throw new InvalidInputException("This sale has already been processed and finalized.");
165            }
166            if (saleDetails.isEmpty()) {
167                throw new InvalidInputException("Cannot process a sale with no items.");
168            }
169
170            // --- BUSINESS RULE 1: Stock verification before confirming sale ---
171            for (SaleDetail detail : saleDetails) {
172                Product prod = detail.getProduct();
173                if (prod.getStockQuantity() < detail.getQuantity()) {
174                    throw new InsufficientStockException(prod.getProductId(), detail.getQuantity(), prod.getStockQuantity());
175                }
176            }
177
178            // Calculate gross total (before discount and tax)
179            this.grossTotal = 0.0;
180            for (SaleDetail detail : saleDetails) {
181                this.grossTotal += detail.getGrossAmount();
182            }
183
184            // Record initial customer reward points
185            this.pointsBeforeSale = customer.getRewardPoints();
186
187            // --- BUSINESS RULE 3 & 4: Polymorphic discount and points handling ---
188            this.discountAmount = customer.calculateDiscount(this.grossTotal);
189            this.pointsRedeemed = customer.redeemPointsForDiscount(this.discountAmount);
190
191            // Subtotal after discount, before tax
192            this.subtotalAfterDiscount = Math.max(0.0, this.grossTotal - this.discountAmount);
193
194            // Calculate tax based on proportion after discount
195            double discountRatio = (this.grossTotal > 0.0) ? (this.subtotalAfterDiscount / this.grossTotal) : 1.0;
196            this.totalTaxAmount = 0.0;
197            for (SaleDetail detail : saleDetails) {
198                double lineGross = detail.getGrossAmount();
199                double lineTaxable = lineGross * discountRatio;
200                double lineTax = (lineTaxable * detail.getTaxRate()) / 100.0;
201                this.totalTaxAmount += lineTax;
202            }
203
204            // Net total payable
205            this.netTotal = this.subtotalAfterDiscount + this.totalTaxAmount;
206
207            // --- BUSINESS RULE 2: Immediate stock deduction ---
208            for (SaleDetail detail : saleDetails) {
209                detail.getProduct().deductStock(detail.getQuantity());
210            }
211
212            // --- BUSINESS RULE 3: Reward points earning for Premium Customers ---
213            // Earns 1 point per Rs. 100 spent (after discounts, before tax)
214            this.pointsEarned = customer.calculatePointsEarned(this.subtotalAfterDiscount);
215            customer.addRewardPoints(this.pointsEarned);
216            this.pointsAfterSale = customer.getRewardPoints();
217
218            this.processed = true;
219        }
220
221        /**
222         * Generates a fully formatted sale bill meeting all Expected Output criteria:
223         * - Product list, quantities, prices, taxes, discounts, total amount,
224         * - Updated reward points (for premium customers),
225         * - Updated stock after sale.
226         */
227        public String getSaleBill() {
228            DateTimeFormatter dtf = DateTimeFormatter.ofPattern("dd-MMM-yyyy HH:mm:ss");
229            StringBuilder sb = new StringBuilder();
230
231            sb.append("=============================================================================================\n");
232            sb.append("                                TAX INVOICE / SALE BILL                                      \n");
233            sb.append("=============================================================================================\n");
234            sb.append(String.format("Invoice No : %-30s Date     : %s\n", saleId, saleDate.format(dtf)));
235            sb.append(String.format("Customer   : %-30s Type     : %s\n", customer.getName(), customer.getCustomerType()));
236            sb.append(String.format("Cust ID    : %-30s Contact  : %s\n", customer.getCustomerId(),
237 customer.getContactNumber()));
238            sb.append("---------------------------------------------------------------------------------------------\n");
            sb.append(String.format("%-8s %-22s %6s %10s %8s %11s %14s\n",
```

(continued) com.inventory.model.Sale.java

```
                "Item ID", "Product Description", "Qty", "Price(Rs. )", "GST %", "Tax(Rs. )", "Line Total(Rs. )"));
        sb.append("-----------------------------------------------------------------------------------------------\n");

        double discountRatio = (this.grossTotal > 0.0) ? (this.subtotalAfterDiscount / this.grossTotal) : 1.0;
        for (SaleDetail d : saleDetails) {
            double lineTaxable = d.getGrossAmount() * discountRatio;
            double itemTax = (lineTaxable * d.getTaxRate()) / 100.0;
            double lineTotalWithTax = lineTaxable + itemTax;

            sb.append(String.format("%-8s %-22s %6d %10.2f %7.1f%% %11.2f %14.2f\n",
                    d.getProduct().getProductId(),
                    d.getProduct().getName(),
                    d.getQuantity(),
                    d.getUnitPrice(),
                    d.getTaxRate(),
                    itemTax,
                    lineTotalWithTax));
        }

        sb.append("-----------------------------------------------------------------------------------------------\n");
        sb.append(String.format("Gross Total (Before Discount & Tax) :                          Rs. %12.2f\n",
grossTotal));
        if (discountAmount > 0.0) {
            sb.append(String.format("Loyalty Discount Applied (%d pts)  :                            -Rs. %12.2f\n",
pointsRedeemed, discountAmount));
            sb.append(String.format("Subtotal (After Discount, Pre-Tax)  :                            Rs. %12.2f\n",
subtotalAfterDiscount));
        }

        sb.append(String.format("Applicable Goods & Services Tax (GST):                        Rs. %12.2f\n",
totalTaxAmount));
        sb.append("-----------------------------------------------------------------------------------------------\n");
        sb.append(String.format("NET TOTAL AMOUNT PAYABLE           :                           Rs. %12.2f\n",
netTotal));
        sb.append("===============================================================================================\n");

        // Reward points summary for premium customers (Rule 3 & 4)
        if (customer instanceof PremiumCustomer) {
            sb.append("REWARD POINTS SUMMARY (PREMIUM CUSTOMER):\n");
            sb.append(String.format("  • Opening Points Balance     : %d pts\n", pointsBeforeSale));
            if (pointsRedeemed > 0) {
                sb.append(String.format("  • Points Redeemed in Invoice : -%d pts (Rs. %.2f discount)\n",
pointsRedeemed, discountAmount));
            }
            sb.append(String.format("  • Points Earned from Purchase: +%d pts (1 pt per Rs. 100 spent)\n",
pointsEarned));
            sb.append(String.format("  • Closing Reward Balance     : %d pts\n", pointsAfterSale));
            sb.append("-----------------------------------------------------------------------------------------------\n");
        } else {
            sb.append("CUSTOMER REWARD STATUS:\n");
            sb.append("  • Regular Customer (No reward points or loyalty discounts applicable)\n");
            sb.append("-----------------------------------------------------------------------------------------------\n");
        }

        // Updated Stock after sale
        sb.append("UPDATED INVENTORY STOCK STATUS (AFTER SALE):\n");
        for (SaleDetail d : saleDetails) {
            sb.append(String.format("  • %-22s (%s) : Remaining Stock = %d units\n",
                    d.getProduct().getName(),
                    d.getProduct().getProductId(),
                    d.getProduct().getStockQuantity()));
        }
        sb.append("===============================================================================================\n");

        return sb.toString();
    }
}
```

[Eclipse IDE Java Editor] Package: com.inventory.exception | File: InsufficientStockException.java

```
package com.inventory.exception;

/**
 * Thrown when a sale cannot proceed due to insufficient stock in the inventory.
 * Enforces Business Rule 1: Stock must be verified before confirming a sale.
 */
public class InsufficientStockException extends Exception {
    private static final long serialVersionUID = 1L;

    private final String productId;
    private final int requestedQuantity;
    private final int availableQuantity;

    public InsufficientStockException(String message) {
        super(message);
        this.productId = "";
        this.requestedQuantity = 0;
```

(continued) com.inventory.exception.InsufficientStockException.java

```
        this.availableQuantity = 0;
    }

    public InsufficientStockException(String productId, int requestedQuantity, int availableQuantity) {
        super("Insufficient stock for Product ID [" + productId + "]. Requested: "
                + requestedQuantity + ", Available: " + availableQuantity);
        this.productId = productId;
        this.requestedQuantity = requestedQuantity;
        this.availableQuantity = availableQuantity;
    }

    public String getProductId() {
        return productId;
    }

    public int getRequestedQuantity() {
        return requestedQuantity;
    }

    public int getAvailableQuantity() {
        return availableQuantity;
    }
}

```

[Eclipse IDE Java Editor] Package: com.inventory.exception | File: ProductNotFoundException.java

```
package com.inventory.exception;

/**
 * Thrown when an operation references a product identifier that does not exist in the inventory.
 */
public class ProductNotFoundException extends Exception {
    private static final long serialVersionUID = 1L;

    public ProductNotFoundException(String message) {
        super(message);
    }

    public ProductNotFoundException(String productId, boolean isId) {
        super("Product with ID '" + productId + "' was not found in the inventory system.");
    }
}

```

[Eclipse IDE Java Editor] Package: com.inventory.exception | File: CustomerNotFoundException.java

```
package com.inventory.exception;

/**
 * Thrown when an operation references a customer ID that does not exist.
 */
public class CustomerNotFoundException extends Exception {
    private static final long serialVersionUID = 1L;

    public CustomerNotFoundException(String message) {
        super(message);
    }

    public CustomerNotFoundException(String customerId, boolean isId) {
        super("Customer with ID '" + customerId + "' was not found in the system.");
    }
}

```

[Eclipse IDE Java Editor] Package: com.inventory.exception | File: DuplicateEntityException.java

```
package com.inventory.exception;

/**
 * Thrown when attempting to register an entity with an identifier that already exists.
 */
public class DuplicateEntityException extends Exception {
    private static final long serialVersionUID = 1L;

    public DuplicateEntityException(String message) {
        super(message);
    }
}

```

[Eclipse IDE Java Editor] Package: com.inventory.exception | File: InvalidInputException.java

```java
package com.inventory.exception;

/**
 * Thrown when an invalid value is supplied by the user during data entry or processing.
 */
public class InvalidInputException extends Exception {
    private static final long serialVersionUID = 1L;

    public InvalidInputException(String message) {
        super(message);
    }
}

```

[Eclipse IDE Java Editor] Package: com.inventory.service | File: InventoryService.java

```java
package com.inventory.service;

import com.inventory.exception.DuplicateEntityException;
import com.inventory.exception.InvalidInputException;
import com.inventory.exception.ProductNotFoundException;
import com.inventory.model.Product;
import com.inventory.model.ProductTaxCategory;
import com.inventory.model.Purchase;
import java.util.ArrayList;
import java.util.Collections;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

/**
 * Service handling inventory management, product catalog, and supplier purchases.
 */
public class InventoryService {
    private final Map<String, Product> productCatalog = new LinkedHashMap<>();
    private final List<Purchase> purchaseHistory = new ArrayList<>();

    public InventoryService() {
        seedInitialProducts();
    }

    /**
     * Seeds initial standard products for immediate testing.
     */
    private void seedInitialProducts() {
        try {
            addProduct(new Product("P101", "Wireless Gaming Mouse", 1499.00, 30, ProductTaxCategory.ELECTRONICS));
            addProduct(new Product("P102", "Mechanical Keyboard", 3499.00, 20, ProductTaxCategory.ELECTRONICS));
            addProduct(new Product("P103", "USB-C Fast Charger 65W", 1299.00, 25, ProductTaxCategory.ELECTRONICS));
            addProduct(new Product("P104", "Organic Green Tea 250g", 350.00, 50, ProductTaxCategory.ESSENTIAL));
            addProduct(new Product("P105", "Executive Diary & Pen", 450.00, 40, ProductTaxCategory.STANDARD));
            addProduct(new Product("P106", "Ergonomic Desk Chair", 8999.00, 10, ProductTaxCategory.LUXURY));
        } catch (Exception ignored) {
        }
    }

    /**
     * Adds a new product to the catalog.
     */
    public synchronized void addProduct(Product product) throws InvalidInputException, DuplicateEntityException {
        if (product == null) {
            throw new InvalidInputException("Product cannot be null.");
        }
        String id = product.getProductId().toUpperCase();
        if (productCatalog.containsKey(id)) {
            throw new DuplicateEntityException("Product with ID '" + id + "' already exists in inventory.");
        }
        productCatalog.put(id, product);
    }

    /**
     * Finds a product by its unique product ID.
     */
    public Product getProductById(String productId) throws ProductNotFoundException {
        if (productId == null || productId.trim().isEmpty()) {
            throw new ProductNotFoundException("Product ID cannot be empty.");
        }
        Product product = productCatalog.get(productId.trim().toUpperCase());
        if (product == null) {
            throw new ProductNotFoundException(productId.trim().toUpperCase(), true);
        }
        return product;
```

(continued) com.inventory.service.InventoryService.java

```
    }

    /**
     * Returns an unmodifiable list of all products in the catalog.
     */
    public List<Product> getAllProducts() {
        return Collections.unmodifiableList(new ArrayList<>(productCatalog.values()));
    }

    /**
     * Records a purchase/restocking event and updates inventory immediately (Rule 2).
     */
    public synchronized void recordPurchase(Purchase purchase) throws InvalidInputException {
        if (purchase == null) {
            throw new InvalidInputException("Purchase transaction cannot be null.");
        }
        purchase.processPurchase();
        purchaseHistory.add(purchase);
    }

    public List<Purchase> getPurchaseHistory() {
        return Collections.unmodifiableList(purchaseHistory);
    }

    /**
     * Displays a clean formatted table of current inventory products.
     */
    public void displayProductsTable() {
        System.out.println("\n" + "=".repeat(92));
        System.out.println("                                CURRENT INVENTORY PRODUCTS                                ");
        System.out.println("=".repeat(92));
        System.out.printf("%-8s | %-26s | %-12s | %-10s | %-22s\n",
                "Prod ID", "Product Name", "Price (Rs. )", "Stock Qty", "Tax Category & Rate");
        System.out.println("-".repeat(92));
        if (productCatalog.isEmpty()) {
            System.out.println("No products currently in stock.");
        } else {
            for (Product p : productCatalog.values()) {
                System.out.printf("%-8s | %-26s | Rs. %-11.2f | %-10d | %-22s\n",
                        p.getProductId(),
                        p.getName(),
                        p.getPrice(),
                        p.getStockQuantity(),
                        p.getTaxCategory().toString());
            }
        }
        System.out.println("=".repeat(92));
    }
}

```

[Eclipse IDE Java Editor] Package: com.inventory.service | File: BillingService.java

```
package com.inventory.service;

import com.inventory.exception.CustomerNotFoundException;
import com.inventory.exception.DuplicateEntityException;
import com.inventory.exception.InsufficientStockException;
import com.inventory.exception.InvalidInputException;
import com.inventory.model.Customer;
import com.inventory.model.PremiumCustomer;
import com.inventory.model.Sale;
import java.util.ArrayList;
import java.util.Collections;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

/**
 * Service handling customer management, billing transactions, and sales records.
 */
public class BillingService {
    private final Map<String, Customer> customerRegistry = new LinkedHashMap<>();
    private final List<Sale> salesHistory = new ArrayList<>();

    public BillingService() {
        seedInitialCustomers();
    }

    /**
     * Seeds initial regular and premium customers for realistic demonstration.
     */
```

(continued) com.inventory.service.BillingService.java

```
30      private void seedInitialCustomers() {
31          try {
32              addCustomer(new Customer("C101", "Rajesh Sharma", "9876543210", "rajesh.sharma@example.com"));
33              addCustomer(new PremiumCustomer("C102", "Priya Sundaram", "9845123456", "priya.sundaram@example.com", 250));
34              addCustomer(new PremiumCustomer("C103", "Anand Kumar", "9765432109", "anand.kumar@example.com", 150));
35              addCustomer(new Customer("C104", "Kavitha Ramesh", "9123456780", "kavitha.r@example.com"));
36          } catch (Exception ignored) {
37          }
38      }
39
40      /**
41       * Registers a customer into the system.
42       */
43      public synchronized void addCustomer(Customer customer) throws InvalidInputException, DuplicateEntityException {
44          if (customer == null) {
45              throw new InvalidInputException("Customer cannot be null.");
46          }
47          String id = customer.getCustomerId().toUpperCase();
48          if (customerRegistry.containsKey(id)) {
49              throw new DuplicateEntityException("Customer with ID '" + id + "' already exists.");
50          }
51          customerRegistry.put(id, customer);
52      }
53
54      /**
55       * Retrieves a customer by ID.
56       */
57      public Customer getCustomerById(String customerId) throws CustomerNotFoundException {
58          if (customerId == null || customerId.trim().isEmpty()) {
59              throw new CustomerNotFoundException("Customer ID cannot be empty.");
60          }
61          Customer customer = customerRegistry.get(customerId.trim().toUpperCase());
62          if (customer == null) {
63              throw new CustomerNotFoundException(customerId.trim().toUpperCase(), true);
64          }
65          return customer;
66      }
67
68      /**
69       * Returns an unmodifiable list of registered customers.
70       */
71      public List<Customer> getAllCustomers() {
72          return Collections.unmodifiableList(new ArrayList<>(customerRegistry.values()));
73      }
74
75      /**
76       * Processes a sale invoice, validates business rules, and saves to history.
77       */
78      public synchronized void recordSale(Sale sale) throws InsufficientStockException, InvalidInputException {
79          if (sale == null) {
80              throw new InvalidInputException("Sale transaction cannot be null.");
81          }
82          sale.processSale();
83          salesHistory.add(sale);
84      }
85
86      /**
87       * Returns the sales history.
88       */
89      public List<Sale> getSalesHistory() {
90          return Collections.unmodifiableList(salesHistory);
91      }
92
93      /**
94       * Displays a formatted table of all registered customers.
95       */
96      public void displayCustomersTable() {
97          System.out.println("\n" + "=".repeat(92));
98          System.out.println("                                REGISTERED CUSTOMERS                                ");
99          System.out.println("=".repeat(92));
100         System.out.printf("%-8s | %-20s | %-12s | %-18s | %-12s\n",
101                 "Cust ID", "Customer Name", "Contact No", "Customer Type", "Points Balance");
102         System.out.println("-".repeat(92));
103         if (customerRegistry.isEmpty()) {
104             System.out.println("No customers currently registered.");
105         } else {
106             for (Customer c : customerRegistry.values()) {
107                 String pointsStr = (c instanceof PremiumCustomer) ? String.valueOf(c.getRewardPoints()) + " pts" : "N/A
(Regular)";     System.out.printf("%-8s | %-20s | %-12s | %-18s | %-12s\n",
108                         c.getCustomerId(),
109                         c.getName(),
110                         c.getContactNumber(),
111
```

(continued) com.inventory.service.BillingService.java

```
                        c.getCustomerType(),
                        pointsStr);
            }
        }
        System.out.println("=".repeat(92));
    }
}

```

[Eclipse IDE Java Editor] Package: com.inventory.util | File: InputValidator.java

```
package com.inventory.util;

import com.inventory.exception.InvalidInputException;
import java.util.Scanner;

/**
 * Robust input reader and validator for console applications.
 * Enforces input validation rules and catches parsing errors without crashing the program.
 */
public class InputValidator {
    private final Scanner scanner;

    public InputValidator(Scanner scanner) {
        this.scanner = scanner;
    }

    /**
     * Reads a non-empty string prompt from the user.
     */
    public String readString(String prompt) {
        while (true) {
            System.out.print(prompt);
            String input = scanner.nextLine();
            if (input != null && !input.trim().isEmpty()) {
                return input.trim();
            }
            System.out.println("  [Validation Error] Input cannot be blank. Please try again.");
        }
    }

    /**
     * Reads an integer within an inclusive range.
     */
    public int readInt(String prompt, int min, int max) {
        while (true) {
            System.out.print(prompt);
            String line = scanner.nextLine();
            try {
                if (line == null || line.trim().isEmpty()) {
                    throw new InvalidInputException("Input cannot be empty.");
                }
                int value = Integer.parseInt(line.trim());
                if (value < min || value > max) {
                    throw new InvalidInputException("Value must be between " + min + " and " + max + ".");
                }
                return value;
            } catch (NumberFormatException e) {
                System.out.println("  [Validation Error] Invalid format! Please enter a valid whole number.");
            } catch (InvalidInputException e) {
                System.out.println("  [Validation Error] " + e.getMessage());
            }
        }
    }

    /**
     * Reads a positive integer (e.g. for quantity).
     */
    public int readPositiveInt(String prompt) {
        return readInt(prompt, 1, Integer.MAX_VALUE);
    }

    /**
     * Reads a non-negative integer (e.g. for initial points).
     */
    public int readNonNegativeInt(String prompt) {
        return readInt(prompt, 0, Integer.MAX_VALUE);
    }

    /**
     * Reads a double value within an inclusive range.
     */
```

(continued) com.inventory.util.InputValidator.java

```
    public double readDouble(String prompt, double min, double max) {
        while (true) {
            System.out.print(prompt);
            String line = scanner.nextLine();
            try {
                if (line == null || line.trim().isEmpty()) {
                    throw new InvalidInputException("Input cannot be empty.");
                }
                double value = Double.parseDouble(line.trim());
                if (value < min || value > max) {
                    throw new InvalidInputException(String.format("Value must be between %.2f and %.2f.", min, max));
                }
                return value;
            } catch (NumberFormatException e) {
                System.out.println("  [Validation Error] Invalid number format! Please enter a valid numeric value.");
            } catch (InvalidInputException e) {
                System.out.println("  [Validation Error] " + e.getMessage());
            }
        }
    }

    /**
     * Reads a positive double (e.g. for price).
     */
    public double readPositiveDouble(String prompt) {
        return readDouble(prompt, 0.01, Double.MAX_VALUE);
    }

    /**
     * Reads a yes/no confirmation.
     */
    public boolean readBoolean(String prompt) {
        while (true) {
            System.out.print(prompt + " (y/n): ");
            String line = scanner.nextLine();
            if (line != null) {
                String val = line.trim().toLowerCase();
                if (val.equals("y") || val.equals("yes")) return true;
                if (val.equals("n") || val.equals("no")) return false;
            }
            System.out.println("  [Validation Error] Please enter 'y' for Yes or 'n' for No.");
        }
    }
}

```

[Eclipse IDE Java Editor] Package: com.inventory.main | File: InventoryBillingApp.java

```
package com.inventory.main;

import com.inventory.exception.CustomerNotFoundException;
import com.inventory.exception.DuplicateEntityException;
import com.inventory.exception.InsufficientStockException;
import com.inventory.exception.InvalidInputException;
import com.inventory.exception.ProductNotFoundException;
import com.inventory.model.Customer;
import com.inventory.model.PremiumCustomer;
import com.inventory.model.Product;
import com.inventory.model.ProductTaxCategory;
import com.inventory.model.Purchase;
import com.inventory.model.PurchaseDetail;
import com.inventory.model.Sale;
import com.inventory.model.SaleDetail;
import com.inventory.service.BillingService;
import com.inventory.service.InventoryService;
import com.inventory.util.InputValidator;
import java.io.PrintStream;
import java.nio.charset.StandardCharsets;
import java.util.Scanner;

/**
 * Main console application entry point for the Inventory and Billing System.
 * Adheres strictly to Specification 1 requirements:
 * Menu Options: Add Product / Add Customer / Make a Purchase / Make a Sale / Display Products / Exit
 */
public class InventoryBillingApp {

    private final InventoryService inventoryService;
    private final BillingService billingService;
    private final InputValidator validator;
    private int purchaseCounter = 1001;
    private int saleCounter = 2001;
```

(continued) com.inventory.main.InventoryBillingApp.java

```
35
36      public InventoryBillingApp() {
37          this.inventoryService = new InventoryService();
38          this.billingService = new BillingService();
39          this.validator = new InputValidator(new Scanner(System.in));
40      }
41
42      public static void main(String[] args) {
43          try {
44              System.setOut(new PrintStream(System.out, true, StandardCharsets.UTF_8.name()));
45              System.setErr(new PrintStream(System.err, true, StandardCharsets.UTF_8.name()));
46          } catch (Exception ignored) {
47          }
48          InventoryBillingApp app = new InventoryBillingApp();
49          app.run();
50      }
51
52      /**
53       * Main application loop displaying the menu and handling user selection.
54       */
55      public void run() {
56          printHeader();
57
58          boolean running = true;
59          while (running) {
60              displayMenu();
61              int choice = validator.readInt("Select an option (1-6): ", 1, 6);
62
63              System.out.println();
64              switch (choice) {
65                  case 1:
66                      handleAddProduct();
67                      break;
68                  case 2:
69                      handleAddCustomer();
70                      break;
71                  case 3:
72                      handleMakePurchase();
73                      break;
74                  case 4:
75                      handleMakeSale();
76                      break;
77                  case 5:
78                      handleDisplayProducts();
79                      break;
80                  case 6:
81                      running = false;
82                      System.out.println("================================================================================");
83                      System.out.println("  Thank you for using the Inventory and Billing System. Goodbye!");
84                      System.out.println("================================================================================");
85                      break;
86                  default:
87                      System.out.println("[Error] Invalid selection. Please choose an option between 1 and 6.");
88              }
89              System.out.println();
90          }
91      }
92
93      private void printHeader() {
94          System.out.println("================================================================================");
95          System.out.println("            INVENTORY AND BILLING SYSTEM - CONSOLE APPLICATION          ");
96          System.out.println("         Assignment - II: Object-Oriented Programming Mini Project      ");
97          System.out.println("================================================================================");
98      }
99
100     private void displayMenu() {
101         System.out.println("--------------------------------------------------------------------------------");
102         System.out.println("                              MAIN MENU                                         ");
103         System.out.println("--------------------------------------------------------------------------------");
104         System.out.println("  1. Add Product");
105         System.out.println("  2. Add Customer");
106         System.out.println("  3. Make a Purchase");
107         System.out.println("  4. Make a Sale");
108         System.out.println("  5. Display Products");
109         System.out.println("  6. Exit");
110         System.out.println("--------------------------------------------------------------------------------");
111     }
112
113     // ========================================================================
114     // 1. ADD PRODUCT
115     // ========================================================================
116     private void handleAddProduct() {
```

(continued) com.inventory.main.InventoryBillingApp.java

```
117             System.out.println("--- [ADD NEW PRODUCT] ---");
118             try {
119                 String productId = validator.readString("Enter Product ID (e.g. P107): ");
120                 String name = validator.readString("Enter Product Name: ");
121                 double price = validator.readPositiveDouble("Enter Selling Price (Rs. ): ");
122                 int initialStock = validator.readNonNegativeInt("Enter Initial Stock Quantity: ");
123 
124                 System.out.println("\nSelect Tax Category:");
125                 System.out.println("  1. Exempt (0% GST)");
126                 System.out.println("  2. Essential (5% GST)");
127                 System.out.println("  3. Standard (12% GST)");
128                 System.out.println("  4. Electronics (18% GST)");
129                 System.out.println("  5. Luxury (28% GST)");
130                 int taxOption = validator.readInt("Choose Tax Category (1-5): ", 1, 5);
131 
132                 ProductTaxCategory taxCat;
133                 switch (taxOption) {
134                     case 1: taxCat = ProductTaxCategory.EXEMPT; break;
135                     case 2: taxCat = ProductTaxCategory.ESSENTIAL; break;
136                     case 3: taxCat = ProductTaxCategory.STANDARD; break;
137                     case 4: taxCat = ProductTaxCategory.ELECTRONICS; break;
138                     case 5: taxCat = ProductTaxCategory.LUXURY; break;
139                     default: taxCat = ProductTaxCategory.STANDARD; break;
140                 }
141 
142                 Product product = new Product(productId, name, price, initialStock, taxCat);
143                 inventoryService.addProduct(product);
144 
145                 System.out.println("\n[Success] Product added successfully!");
146                 System.out.println("Product Details: " + product);
147 
148             } catch (DuplicateEntityException | InvalidInputException e) {
149                 System.out.println("\n[Error] Unable to add product: " + e.getMessage());
150             } catch (Exception e) {
151                 System.out.println("\n[Unexpected Error] An unexpected error occurred: " + e.getMessage());
152             }
153         }
154 
155         // ======================================================================
156         // 2. ADD CUSTOMER
157         // ======================================================================
158         private void handleAddCustomer() {
159             System.out.println("--- [ADD NEW CUSTOMER] ---");
160             try {
161                 String customerId = validator.readString("Enter Customer ID (e.g. C105): ");
162                 String name = validator.readString("Enter Customer Name: ");
163                 String contact = validator.readString("Enter Contact Number: ");
164                 String email = validator.readString("Enter Email Address: ");
165 
166                 System.out.println("\nSelect Customer Type:");
167                 System.out.println("  1. Regular Customer (Standard billing, no loyalty points)");
168                 System.out.println("  2. Premium Customer (Earns 1 pt/Rs. 100, discounts when pts >= 200)");
169                 int typeChoice = validator.readInt("Choose Customer Type (1-2): ", 1, 2);
170 
171                 Customer customer;
172                 if (typeChoice == 2) {
173                     int initialPoints = validator.readNonNegativeInt("Enter Initial Reward Points: ");
174                     customer = new PremiumCustomer(customerId, name, contact, email, initialPoints);
175                 } else {
176                     customer = new Customer(customerId, name, contact, email);
177                 }
178 
179                 billingService.addCustomer(customer);
180                 System.out.println("\n[Success] " + customer.getCustomerType() + " registered successfully!");
181                 System.out.println("Customer Profile: " + customer);
182 
183             } catch (DuplicateEntityException | InvalidInputException e) {
184                 System.out.println("\n[Error] Registration failed: " + e.getMessage());
185             } catch (Exception e) {
186                 System.out.println("\n[Unexpected Error] An unexpected error occurred: " + e.getMessage());
187             }
188         }
189 
190         // ======================================================================
191         // 3. MAKE A PURCHASE (Restocking from Supplier)
192         // ======================================================================
193         private void handleMakePurchase() {
194             System.out.println("--- [MAKE A PURCHASE / RESTOCK INVENTORY] ---");
195             try {
196                 inventoryService.displayProductsTable();
197 
198                 String purchaseId = "PUR-" + (purchaseCounter++);
```

(continued) com.inventory.main.InventoryBillingApp.java

```
            System.out.println("Generated Purchase ID: " + purchaseId);
            String supplierName = validator.readString("Enter Supplier / Distributor Name: ");

            Purchase purchase = new Purchase(purchaseId, supplierName);

            boolean addMore = true;
            while (addMore) {
                String productId = validator.readString("\nEnter Product ID to restock: ");
                Product product;
                try {
                    product = inventoryService.getProductById(productId);
                } catch (ProductNotFoundException e) {
                    System.out.println("[Error] " + e.getMessage());
                    boolean retry = validator.readBoolean("Would you like to try another product ID?");
                    if (retry) continue;
                    else break;
                }

                System.out.println("Selected Product: " + product.getName() + " (Current Stock: " +
product.getStockQuantity() + ")");
                int quantity = validator.readPositiveInt("Enter Restock Quantity: ");
                double costPrice = validator.readPositiveDouble("Enter Unit Cost Price (Rs. ): ");

                PurchaseDetail detail = new PurchaseDetail(product, quantity, costPrice);
                purchase.addPurchaseDetail(detail);

                addMore = validator.readBoolean("Add another item to this purchase order?");
            }

            if (purchase.getPurchaseDetails().isEmpty()) {
                System.out.println("\n[Notice] No items were added to the purchase order. Transaction aborted.");
                return;
            }

            // Confirm and execute purchase (Enforces Business Rule 2)
            inventoryService.recordPurchase(purchase);

            System.out.println("\n[Success] Purchase order confirmed and inventory stock updated immediately!");
            System.out.println(purchase.getPurchaseReceipt());

        } catch (InvalidInputException e) {
            System.out.println("\n[Error] Purchase transaction failed: " + e.getMessage());
        } catch (Exception e) {
            System.out.println("\n[Unexpected Error] An unexpected error occurred: " + e.getMessage());
        }
    }

    // ========================================================================
    // 4. MAKE A SALE (Billing & Rewards)
    // ========================================================================
    private void handleMakeSale() {
        System.out.println("--- [MAKE A SALE / ISSUE INVOICE] ---");
        try {
            billingService.displayCustomersTable();

            String customerId = validator.readString("Enter Customer ID for billing: ");
            Customer customer;
            try {
                customer = billingService.getCustomerById(customerId);
            } catch (CustomerNotFoundException e) {
                System.out.println("\n[Error] " + e.getMessage());
                System.out.println("Please register the customer using Option 2 before creating a sale.");
                return;
            }

            System.out.println("\nSelected Customer: " + customer.getName() + " (" + customer.getCustomerType() + ")");
            if (customer instanceof PremiumCustomer) {
                System.out.println("Current Reward Points: " + customer.getRewardPoints() + " pts");
                if (customer.getRewardPoints() >= 200) {
                    int discountBlocks = customer.getRewardPoints() / 100;
                    System.out.println("-> Eligible for Rs. " + (discountBlocks * 10) + " loyalty discount ("
                            + (discountBlocks * 100) + " points will be redeemed).");
                } else {
                    System.out.println("-> Points below 200 threshold; no points discount applied today.");
                }
            }

            inventoryService.displayProductsTable();

            String saleId = "INV-" + (saleCounter++);
            Sale sale = new Sale(saleId, customer);

            boolean addMore = true;
```

(continued) com.inventory.main.InventoryBillingApp.java

```
            while (addMore) {
                String productId = validator.readString("\nEnter Product ID to sell: ");
                Product product;
                try {
                    product = inventoryService.getProductById(productId);
                } catch (ProductNotFoundException e) {
                    System.out.println("[Error] " + e.getMessage());
                    boolean retry = validator.readBoolean("Try another product ID?");
                    if (retry) continue;
                    else break;
                }

                System.out.printf("Product: %s | Price: Rs. %.2f | Available Stock: %d\n",
                        product.getName(), product.getPrice(), product.getStockQuantity());

                int quantity = validator.readPositiveInt("Enter Sale Quantity: ");

                // Early check before adding detail
                if (product.getStockQuantity() < quantity) {
                    System.out.printf("  [Warning] Insufficient stock! Available: %d, Requested: %d.\n",
                            product.getStockQuantity(), quantity);
                    boolean tryDifferentQty = validator.readBoolean("Do you want to re-enter quantity for this item?");
                    if (tryDifferentQty) {
                        continue;
                    } else {
                        boolean cont = validator.readBoolean("Add a different product instead?");
                        if (cont) continue;
                        else break;
                    }
                }

                SaleDetail detail = new SaleDetail(product, quantity);
                sale.addSaleDetail(detail);

                addMore = validator.readBoolean("Add another product to this invoice?");
            }

            if (sale.getSaleDetails().isEmpty()) {
                System.out.println("\n[Notice] No products added. Sale invoice cancelled.");
                return;
            }

            // Confirm and execute the sale
            // Enforces Business Rules 1, 2, 3, 4, 5
            billingService.recordSale(sale);

            System.out.println("\n[Success] Sale confirmed and completed successfully!");
            System.out.println(sale.getSaleBill());

        } catch (InsufficientStockException e) {
            System.out.println("\n[Business Rule 1 Violation - Sale Aborted]");
            System.out.println(e.getMessage());
            System.out.println("Action: No stock was deducted. The sale has been terminated.");
        } catch (InvalidInputException e) {
            System.out.println("\n[Error] Sale processing failed: " + e.getMessage());
        } catch (Exception e) {
            System.out.println("\n[Unexpected Error] An unexpected error occurred: " + e.getMessage());
        }
    }

    // ========================================================================
    // 5. DISPLAY PRODUCTS
    // ========================================================================
    private void handleDisplayProducts() {
        inventoryService.displayProductsTable();
    }
}

```

4. EXECUTION OUTPUT (ECLIPSE CONSOLE SIMULATION)

The following console runs simulate the Eclipse Console window. They illustrate menu navigation, catalog display, stock verification, error handling with custom exceptions, restock purchases, and regular/premium billing bills.

Scenario 1: Main Menu & Product Catalog Display (Option 5)

Displays the initial inventory with selling prices, available quantities, and tax categories.

<terminated> InventoryBillingApp [Java Application] Eclipse Temurin OpenJDK 17.0.20 Console

```
======================================================================
           INVENTORY AND BILLING SYSTEM - CONSOLE APPLICATION
         Assignment - II: Object-Oriented Programming Mini Project
======================================================================
----------------------------------------------------------------------
                            MAIN MENU
----------------------------------------------------------------------
  1. Add Product
  2. Add Customer
  3. Make a Purchase
  4. Make a Sale
  5. Display Products
  6. Exit
----------------------------------------------------------------------
Select an option (1-6): 5

=============================================================================================
                              CURRENT INVENTORY PRODUCTS
=============================================================================================
Prod ID  | Product Name              | Price (Rs.)  | Stock Qty  | Tax Category & Rate
---------------------------------------------------------------------------------------------
P101     | Wireless Gaming Mouse     | Rs.1499.00   | 30         | Electronics (18%) [18.0%]
P102     | Mechanical Keyboard       | Rs.3499.00   | 20         | Electronics (18%) [18.0%]
P103     | USB-C Fast Charger 65W    | Rs.1299.00   | 25         | Electronics (18%) [18.0%]
P104     | Organic Green Tea 250g    | Rs.350.00    | 50         | Essential (5%) [5.0%]
P105     | Executive Diary & Pen     | Rs.450.00    | 40         | Standard (12%) [12.0%]
P106     | Ergonomic Desk Chair      | Rs.8999.00   | 10         | Luxury (28%) [28.0%]
=============================================================================================
```

Scenario 2: Add New Product & Add New Customer (Options 1 & 2)

Registers a new electronics item and a Premium Customer with initial reward points.

<terminated> InventoryBillingApp [Java Application] Eclipse Temurin OpenJDK 17.0.20 Console

```
Select an option (1-6): 1
--- [ADD NEW PRODUCT] ---
Enter Product ID (e.g. P107): P107
Enter Product Name: Noise-Cancelling Headphones
Enter Selling Price (Rs.): 4999.00
Enter Initial Stock Quantity: 15

Select Tax Category:
  1. Exempt (0% GST)
  2. Essential (5% GST)
  3. Standard (12% GST)
  4. Electronics (18% GST)
  5. Luxury (28% GST)
Choose Tax Category (1-5): 4

[Success] Product added successfully!
Product Details: P107     | Noise-Cancelling Headphones | Rs.4999.00   | Stock: 15    | Tax: Electronics (18%) [18.0%]

Select an option (1-6): 2
--- [ADD NEW CUSTOMER] ---
Enter Customer ID (e.g. C105): C105
Enter Customer Name: Sneha Patel
Enter Contact Number: 9988776655
Enter Email Address: sneha.p@example.com

Select Customer Type:
  1. Regular Customer (Standard billing, no loyalty points)
  2. Premium Customer (Earns 1 pt/Rs.100, discounts when pts >= 200)
Choose Customer Type (1-2): 2
Enter Initial Reward Points: 350

[Success] Premium (Silver) registered successfully!
Customer Profile: C105     | Sneha Patel          | 9988776655   | Premium    | sneha.p@example.com | Points: 350
```

Scenario 3: Make a Purchase & Immediate Stock Update (Option 3 - Rule 2)

Restocks Product P101 from supplier. Confirms immediate stock quantity update from 30 to 40.

<terminated> InventoryBillingApp [Java Application] Eclipse Temurin OpenJDK 17.0.20 Console

```
Select an option (1-6): 3
--- [MAKE A PURCHASE / RESTOCK INVENTORY] ---
Generated Purchase ID: PUR-1001
Enter Supplier / Distributor Name: Wholesale Tech Supplies

Enter Product ID to restock: P101
Selected Product: Wireless Gaming Mouse (Current Stock: 30)
Enter Restock Quantity: 10
Enter Unit Cost Price (Rs.): 950.00
Add another item to this purchase order? (y/n): n

[Success] Purchase order confirmed and inventory stock updated immediately!
========================================================================
                    PURCHASE / RESTOCK RECEIPT
========================================================================
Purchase ID : PUR-1001                 Date: 25-Oct-2026 10:31:07
Supplier    : Wholesale Tech Supplies
------------------------------------------------------------------------
Prod ID    Product Name                  Qty   Cost (Rs.) Subtotal (Rs.)
------------------------------------------------------------------------
P101       Wireless Gaming Mouse          10       950.00        9500.00
------------------------------------------------------------------------
TOTAL PURCHASE AMOUNT:                                 Rs.       9500.00
------------------------------------------------------------------------
UPDATED PRODUCT STOCK LEVELS:
  • Wireless Gaming Mouse (P101): Current Available Stock = 40 units
========================================================================
```

Scenario 4: Sale to Regular Customer (Option 4 - Rule 4)

Regular customer Rajesh Sharma (C101). Confirms zero discount and zero reward points accrued.

<terminated> InventoryBillingApp [Java Application] Eclipse Temurin OpenJDK 17.0.20 Console

```
Select an option (1-6): 4
--- [MAKE A SALE / ISSUE INVOICE] ---
Enter Customer ID for billing: C101
Selected Customer: Rajesh Sharma (Regular)

Enter Product ID to sell: P104
Product: Organic Green Tea 250g | Price: Rs.350.00 | Available Stock: 50
Enter Sale Quantity: 2
Add another product to this invoice? (y/n): n

[Success] Sale confirmed and completed successfully!
=============================================================================================
                              TAX INVOICE / SALE BILL
=============================================================================================
Invoice No : INV-2001                      Date     : 25-Oct-2026 10:31:07
Customer   : Rajesh Sharma                 Type     : Regular
Cust ID    : C101                          Contact  : 9876543210
---------------------------------------------------------------------------------------------
Item ID  Product Description       Qty Price(Rs.)    GST %    Tax(Rs.) Line Total(Rs.)
---------------------------------------------------------------------------------------------
P104     Organic Green Tea 250g      2     350.00     5.0%       35.00         735.00
---------------------------------------------------------------------------------------------
Gross Total (Before Discount & Tax) :                            Rs.      700.00
Applicable Goods & Services Tax (GST):                           Rs.       35.00
---------------------------------------------------------------------------------------------
NET TOTAL AMOUNT PAYABLE           :                             Rs.      735.00
=============================================================================================
CUSTOMER REWARD STATUS:
  • Regular Customer (No reward points or loyalty discounts applicable)
---------------------------------------------------------------------------------------------
UPDATED INVENTORY STOCK STATUS (AFTER SALE):
  • Organic Green Tea 250g (P104) : Remaining Stock = 48 units
=============================================================================================
```

Scenario 5: Sale to Premium Customer (Option 4 - Rule 3)

Premium customer Priya Sundaram (C102, 250 pts). Redaction of 200 pts for Rs.20 discount, plus accrual of 64 new pts.

<terminated> InventoryBillingApp [Java Application] Eclipse Temurin OpenJDK 17.0.20 Console

```
Select an option (1-6): 4
--- [MAKE A SALE / ISSUE INVOICE] ---
Enter Customer ID for billing: C102
Selected Customer: Priya Sundaram (Premium (Silver))
Current Reward Points: 250 pts
-> Eligible for Rs.20 loyalty discount (200 points will be redeemed).

Enter Product ID to sell: P102
Product: Mechanical Keyboard | Price: Rs.3499.00 | Available Stock: 20
Enter Sale Quantity: 1
Add another product to this invoice? (y/n): y

Enter Product ID to sell: P101
Product: Wireless Gaming Mouse | Price: Rs.1499.00 | Available Stock: 40
Enter Sale Quantity: 2
Add another product to this invoice? (y/n): n

[Success] Sale confirmed and completed successfully!
================================================================================
                          TAX INVOICE / SALE BILL
================================================================================
Invoice No : INV-2002                    Date     : 25-Oct-2026 10:31:07
Customer   : Priya Sundaram              Type     : Premium (Silver)
Cust ID    : C102                        Contact  : 9845123456
--------------------------------------------------------------------------------
Item ID  Product Description       Qty Price(Rs.)    GST %    Tax(Rs.) Line Total(Rs.)
--------------------------------------------------------------------------------
P102     Mechanical Keyboard         1    3499.00    18.0%      627.88        4116.11
P101     Wireless Gaming Mouse       2    1499.00    18.0%      537.98        3526.75
--------------------------------------------------------------------------------
Gross Total (Before Discount & Tax) :                          Rs.     6497.00
Loyalty Discount Applied (200 pts)  :                         -Rs.       20.00
Subtotal (After Discount, Pre-Tax)  :                          Rs.     6477.00
Applicable Goods & Services Tax (GST):                         Rs.     1165.86
--------------------------------------------------------------------------------
NET TOTAL AMOUNT PAYABLE            :                          Rs.     7642.86
================================================================================
REWARD POINTS SUMMARY (PREMIUM CUSTOMER):
  • Opening Points Balance     : 250 pts
  • Points Redeemed in Invoice : -200 pts (Rs.20.00 discount)
  • Points Earned from Purchase: +64 pts (1 pt per Rs.100 spent)
  • Closing Reward Balance     : 114 pts
--------------------------------------------------------------------------------
UPDATED INVENTORY STOCK STATUS (AFTER SALE):
  • Mechanical Keyboard    (P102) : Remaining Stock = 19 units
  • Wireless Gaming Mouse  (P101) : Remaining Stock = 38 units
================================================================================
```

Scenario 6: Stock Verification & Exception Handling (Rule 1)

Customer attempts to buy 99 units of P106 with only 10 available. The application catches stock deficit without crashing.

<terminated> InventoryBillingApp [Java Application] Eclipse Temurin OpenJDK 17.0.20 Console

```
Select an option (1-6): 4
--- [MAKE A SALE / ISSUE INVOICE] ---
Enter Customer ID for billing: C101
Selected Customer: Rajesh Sharma (Regular)

Enter Product ID to sell: P106
Product: Ergonomic Desk Chair | Price: Rs.8999.00 | Available Stock: 10
Enter Sale Quantity: 99
  [Warning] Insufficient stock! Available: 10, Requested: 99.
Do you want to re-enter quantity for this item? (y/n): n
Add a different product instead? (y/n): n

[Notice] No products added. Sale invoice cancelled.
Action: No stock was deducted. The transaction was aborted cleanly without crashing.
```

4. GitHub Repository Link

GitHub Repository URL:

https://github.com/tcgmegavarsan-sys/Java-Programming-Mini-Project-Assignment-II

Student: MEGA VARSAN VE | Roll No / Reg. No: 717825I231 | Dept: AD (Section B)

Repository contains complete source code, Eclipse IDE project files (.project, .classpath), executable bytecode, execution logs, and documentation README.md.